

Introduction to Logistic Regression

- Logistic Regression is a **supervised learning** algorithm used for **classification problems**.
- It predicts the **probability** of an outcome belonging to a specific class.
- Unlike Linear Regression, it uses a **sigmoid function** to restrict output between **0 and 1**.
- **Key Assumption:** The dependent variable is binary (0 or 1) for standard logistic regression.
- **Example Applications:**
 - Determining whether a patient has **diabetes (Yes/No)**.
 - Identifying if an email is **spam or not spam**.
 - Predicting whether a student will **pass or fail** an exam.

Types of Logistic Regression

1. **Binary Logistic Regression:** Two possible outcomes (e.g., **spam vs. not spam**).
2. **Multinomial Logistic Regression:** More than two categories **without** ordering (e.g., **predicting blood type: A, B, AB, or O**).
3. **Ordinal Logistic Regression:** More than two categories **with** ordering (e.g., **customer satisfaction levels: Low, Medium, High**).

Why Not Linear Regression?

- **Linear Regression Limitation:**
 - It does not constrain the predicted values between **0 and 1**.
 - It assumes a **linear** relationship between variables.
 - It is **not robust** to outliers in classification tasks.
- **Logistic Regression Solution:**
 - Uses the **sigmoid function** to squash values between **0 and 1**.
 - Provides **probabilistic interpretation**.
 - Outputs a **decision boundary** instead of continuous values.
- **What is the Sigmoid Function?**

The **sigmoid function** is a mathematical function that maps any real-valued number to a value between **0 and 1**. It is often used in logistic regression and neural networks to model probabilities.

Mathematical Definition:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where:

- $z = \theta^T x$ (linear combination of weights and inputs)
- e is Euler's number (≈ 2.718)

Graphical Representation:

- The sigmoid curve is S-shaped (also called a logistic curve).
- It asymptotically approaches 0 on the left and 1 on the right.
- When $z = 0$, the output is 0.5.

**Why do we use sigmoid function?**

- The sigmoid function is used because:
 - It maps any real number to the range (0,1), which is suitable for probability estimation.
 - It is differentiable, allowing optimization using gradient descent.
 - It has a smooth S-shape, ensuring gradual transitions between classes.
 - It helps define a decision boundary based on probability thresholds.
- Mathematical Definition: $\sigma(z) = \frac{1}{1+e^{-z}}$
- Example Calculation:
 - Given $z = 2$, $\sigma(2) = \frac{1}{1+e^{-2}} = 0.88$.
 - Given $z = -3$, $\sigma(-3) = \frac{1}{1+e^3} = 0.047$.
- Graphical Representation: Sigmoid function curve with limits at 0 and 1.

Example Calculation:

If $z = 2$, the sigmoid function gives:

$$\sigma(2) = \frac{1}{1 + e^{-2}} \approx \frac{1}{1 + 0.135} = 0.88$$

This means that the model assigns **88% probability** to class 1.

Logistic Regression Hypothesis Function

- The logistic regression model uses: $h_{\theta}(x) = \sigma(\theta^T x)$
- Parameters (θ): Learned during training.
- Decision Rule:
 - If $h(x) \geq 0.5$, classify as 1.
 - If $h(x) < 0.5$, classify as 0.
- Example: Predicting whether a customer will purchase a product based on income and previous purchase history.

Decision boundary

- Defined by the equation: $\theta^T x = 0$.
- Linear Decision Boundary: When features are linearly separable.
- Non-Linear Decision Boundary: Requires feature transformations.
- Example: Predicting whether an applicant qualifies for a loan based on income and credit score.

Cost Function for logistic Regression

- Instead of Mean Squared Error (MSE), Logistic Regression uses
- Log Loss: $J(\theta) = -\frac{1}{m} \sum [y \log h_{\theta}(x) + (1 - y) \log(1 - h_{\theta}(x))]$
- The cost function is **convex**, making optimization feasible.
- Log loss penalizes wrong predictions more heavily.

Gradient Decent Optimization

- Updates parameters iteratively: $\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$
- Learning rate (α *****): Controls step size in optimization.
- Stopping Criterion: When cost function stabilizes.

Overfitting Legalization

- **Overfitting:** Model memorizes noise instead of patterns.
- **Regularization Techniques:**
 - **L1 (Lasso):** Shrinks coefficients, leading to feature selection.
 - **L2 (Ridge):** Reduces coefficient magnitudes, preventing extreme values.
- **Regularized Cost Function:** $J(\theta) + \lambda \sum \theta^2$

Multiclass Logistic Regression

- **One-vs-All (OvA):** Trains multiple binary classifiers.
- **Softmax Regression:** Assigns probabilities to multiple classes.
- Example: Predicting the **digit (0-9) in handwritten recognition**.

Performance Metrics

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** $\frac{TP}{TP+FP}$
- **Recall:** $\frac{TP}{TP+FN}$
- **F1-Score:** Harmonic mean of precision and recall.

Medical Diagnosis

- Predicting if a patient has diabetes based on glucose levels.
- **Given:**
 - Age = 50, BMI = 30, Glucose = 140
- **Model Prediction:** $z = -3.2 + (0.02 \times 50) + (0.1 \times 30) + (0.05 \times 140) = 1.8$ $P(\text{Diabetes}) = \sigma(1.8) = 0.858$
- Since $P > 0.5$, classify as **Diabetic**.

Example

Example: Predicting Employee Promotion Using Logistic Regression & Gradient Descent.

Step 1: Define the Problem

We have the following dataset:

Years of Experience (x)	Promotion (y) (0 = No, 1 = Yes)
1	0
3	0
5	1
7	1
9	1

We want to use logistic regression to find the probability that an employee gets a promotion.

Step 2: Define the Hypothesis (Sigmoid Function)

Logistic regression uses the sigmoid function:

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}}$$

where:

$$z = \theta_0 + \theta_1 x$$

- θ_0 and θ_1 are the parameters we need to find.
- x is the feature (years of experience).

Step 3: Define the Cost Function

The cost function for logistic regression is:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

where:

- m is the number of training examples.
- $y^{(i)}$ is the actual label (0 or 1).
- $h_{\theta}(x^{(i)})$ is the predicted probability.

Our goal is to minimize $J(\theta)$ to find the best values of θ_0 and θ_1 .

Step 4: Compute the Gradient

The gradients for updating θ values are:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$
$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$$

These gradients will be used in **Gradient Descent** to update θ .

Step 5: Apply Gradient Descent

The update rules for θ are:

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$
$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

where α is the learning rate.

Example Calculation

Initialize Parameters

Let's start with:

$$\theta_0 = 0, \quad \theta_1 = 0$$

Set learning rate:

$$\alpha = 0.1$$

Compute Predictions

Since $\theta_0 = 0$ and $\theta_1 = 0$, the sigmoid function gives:

$$h_{\theta}(x) = \frac{1}{1 + e^0} = 0.5$$

for all data points.

Compute Gradient

Using:

$$\begin{aligned}\frac{\partial J}{\partial \theta_0} &= \frac{1}{5} \sum (h_{\theta}(x) - y) \\ &= \frac{1}{5} [(0.5 - 0) + (0.5 - 0) + (0.5 - 1) + (0.5 - 1) + (0.5 - 1)] \\ &= \frac{1}{5} [0.5 + 0.5 - 0.5 - 0.5 - 0.5] = -0.1\end{aligned}$$

For θ_1 :

$$\begin{aligned}\frac{\partial J}{\partial \theta_1} &= \frac{1}{5} \sum (h_{\theta}(x) - y)x \\ &= \frac{1}{5} [(0.5 - 0) \cdot 1 + (0.5 - 0) \cdot 3 + (0.5 - 1) \cdot 5 + (0.5 - 1) \cdot 7 + (0.5 - 1) \cdot 9] \\ &= \frac{1}{5} [0.5 + 1.5 - 2.5 - 3.5 - 4.5] = -1.3\end{aligned}$$

Update θ

$$\theta_0 = \theta_0 - \alpha \frac{\partial J}{\partial \theta_0} = 0 - (0.1 \times -0.1) = 0.01$$

$$\theta_1 = \theta_1 - \alpha \frac{\partial J}{\partial \theta_1} = 0 - (0.1 \times -1.3) = 0.13$$

New values:

- $\theta_0 = 0.01$
- $\theta_1 = 0.13$

 Repeat these steps until convergence.



- Gradient descent **iteratively updates** θ values to minimize the cost.
- The **sigmoid function** converts predictions into probabilities.
- By running gradient descent for **multiple iterations**, we get optimal values of θ .

